

Chronic Kidney Disease Prediction Project Notes

This project focuses on predicting Chronic Kidney Disease (CKD) using Machine Learning and Artificial Intelligence models.

The notebook includes data preprocessing, visualization, feature engineering, model training, model comparison, evaluation metrics, and prediction testing using patient data.

Project Workflow

- Dataset collection and loading.
- Data cleaning and preprocessing.
- Handling missing values and encoding.
- Feature scaling using StandardScaler.
- Data visualization using graphs and heatmaps.
- Training multiple Machine Learning models.
- Comparing model performance and evaluation.
- Testing predictions using patient input data.

Detailed Notebook and Soft Copy Explanation

This added section explains what was done inside the notebook, what tools and libraries were used, how the important numbers were chosen, and what should be included in the soft copy that will be uploaded to Drive.

1. What the Notebook Explains

- The notebook starts by installing and importing the required Python libraries used for data handling, visualization, machine learning, and artificial intelligence.
- The dataset is loaded from the project folder as kidney_disease_dataset.csv, then the first rows, data types, missing values, and statistical summary are checked.
- The CKD_Status column is the target column. It was converted from text into numbers so the models can learn from it: kidney healthy = 0 and kidney problem = 1.
- The Id column was removed because it is only an identifier and does not help the model predict kidney disease.
- A new feature called Creatinine_GFR_Ratio was created using $\text{Creatinine_Level} / (\text{GFR} + 1)$. This combines two medically important kidney indicators and helps the models detect kidney problems more clearly.
- The data was split into training and testing sets. The training set is used to teach the models, while the testing set is used to evaluate performance on unseen data.
- The features were scaled using StandardScaler so that all numerical values have a similar range, which is important for models such as ANN, SVC, and KNN.

- Five models were trained and compared: Logistic Regression, Artificial Neural Network (ANN), Support Vector Classifier (SVC), K-Nearest Neighbors (KNN), and Decision Tree.
- At the end of the notebook, a patient input example was tested using all models to show how the system can predict whether a patient is kidney healthy or has a kidney problem.

2. Tools, Libraries, and Their Uses

Tool / Library	How it was used in the project
Python	Main programming language used to build and run the project.
Pandas	Used to load the CSV file and handle the dataset as a DataFrame.
NumPy	Used for numerical operations and array handling.
Matplotlib	Used to display graphs and figures.
Seaborn	Used to create count plots, pair plots, correlation heatmaps, and feature relation graphs.
Scikit-learn	Used for preprocessing, splitting data, scaling, model training, and evaluation metrics.
TensorFlow / Keras	Used to build and train the Artificial Neural Network model.
StandardScaler	Used to standardize features before model training.
LabelEncoder	Used to convert CKD_Status from text labels into numerical labels.

3. Project Data Analysis Summary

- The dataset contains 2198 patient records.
- And the other 107 records for human Testing
- The original dataset contains 10 columns: Id, Age, Creatinine_Level, BUN, Diabetes, Hypertension, GFR, Urine_Output, CKD_Status, and Dialysis_Needed. And other feature will be added
- There were no missing values in the dataset, because every column had 2198 non-null records.
- There were no duplicated rows.
- The target column had two classes: kidney healthy and kidney problem.
- The Id column was removed because it does not represent a medical feature.
- Outliers were checked using the IQR method, but they were not removed because this is medical data and extreme values may represent real clinical cases.
- The outlier check showed 8 outliers in Creatinine_Level, 8 in BUN, 9 in Urine_Output, 31 in Dialysis_Needed, and 111 in Creatinine_GFR_Ratio.
- The visualizations helped show class balance, relationships between features, and which variables are most related to CKD_Status.

4. Why These Numbers Were Chosen

Number / Setting	Reason for choosing it
0 and 1 target encoding	Machine learning models need numerical labels. 0 was used for kidney healthy and 1 for kidney problem.
test_size = 0.3	30% of the data was used for testing and 70% for training to evaluate the model on unseen data while keeping enough data for learning.
GFR + 1 in Creatinine_GFR_Ratio	Adding 1 avoids division by zero and makes the ratio safer to calculate.
IQR rule = 1.5 x IQR	This is a common rule for detecting outliers in numerical data.
ANN hidden layers: 16 and 8 neurons	A small ANN structure was selected to learn nonlinear patterns without making the model too complex.
ANN output sigmoid	Sigmoid is suitable for binary classification because it gives a probability between 0 and 1.
ANN threshold = 0.5	Values above 0.5 are classified as kidney problem; values below or equal 0.5 are classified as kidney healthy.
epochs = 10	The ANN was trained for 10 full passes over the training data, enough for this small project without excessive training time.
batch_size = 16	The data was processed in small batches to make training stable and efficient.
KNN n_neighbors = 5	The prediction is based on the 5 closest records, which is a common starting value for KNN.
Decision Tree criterion = entropy	Entropy uses information gain to choose the best feature splits.
Decision Tree random_state = 51	Used to make the Decision Tree results reproducible when the code is rerun.

5. Goal of the Implementation

- The main goal is to build a simple machine learning system that can predict Chronic Kidney Disease status from patient medical features.
- The project compares several models to identify which one performs best on this dataset.
- The project also shows the full workflow from loading the data to testing the final prediction using a new patient input.
- The final prediction output helps classify a patient as Kidney Healthy or Kidney Problem, but it should be considered an educational machine learning project, not a medical diagnosis system.

6. Explanation of the Final Patient Test

The notebook includes one patient example with the following values: Age = 89, Creatinine_Level = 1.48, BUN = 5.2, Diabetes = 0, Hypertension = 0, GFR = 74.5, Urine_Output = 988, Dialysis_Needed = 0, and Creatinine_GFR_Ratio = $1.48 / (74.5 + 1)$.

All trained models predicted this patient as Kidney Healthy. The ANN probability was 0.03472506, meaning the ANN gave a low probability that this patient has a kidney problem. Because this value is below the 0.5 threshold, the ANN prediction was Kidney Healthy.

Important Observations

- 1-the data didn't have any nulls or duplicates and outliers was not removed as it's a medical data but encoding was done to the CDK_Status column as 1 represents the kidney problem and 0 represents the kidney is healthy
- 2- Feature scaling and splitting improved model stability and performance by calculating the creatinine GFR ratio which equal to $\text{Creatinine_Level} / (\text{GFR} + 1)$ which is used by ANN to detect is there a kidney problem and increase accuracy
- 3- Decision Tree achieved the best overall performance on this dataset.
- 4-ANN and SVC also showed very strong predictive capability.
- 5-Logistic Regression worked well as a baseline model but was less accurate.
- 6-Correlation graphs helped identify the most important features affecting CKD.

Model Comparison

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Logistic Regression	0.8197	0.8034	0.8494	0.8258	0.8195
ANN	0.8985	0.8711	0.9367	0.9028	0.8983
SVC	0.9394	0.9195	0.9639	0.9412	0.9392
KNN	0.8712	0.8946	0.8434	0.8682	0.8714
Decision Tree	1.0	1.0	1.0	1.0	1.0

Pros and Cons of Each Model

Logistic Regression

Pros: Simple, fast, easy to understand, good baseline model.

Cons: Limited for complex nonlinear relationships.

ANN

Pros: Can learn complex patterns and relationships in medical data.

Cons: Needs more data and computational power.

SVC

Pros: Very strong classification performance and good with high-dimensional data.

Cons: Training can be slower with larger datasets.

KNN

Pros: Easy to implement and works well with local data patterns.

Cons: Slow prediction time and sensitive to scaling.

Decision Tree

Pros: Very interpretable and achieved the highest accuracy in this project.

Cons: Can overfit the training data.

Best Model in the Project

The Decision Tree model achieved the highest evaluation metrics with:

- Accuracy = 100%
- Precision = 100%
- Recall = 100%
- F1 Score = 100%
- ROC-AUC = 100%

This indicates that the Decision Tree perfectly classified the test data in this experiment. However, this may also indicate possible overfitting, so further testing on unseen datasets is recommended.

Graphs and Visualizations





